



Laurea Triennale in informatica-Università di Salerno
Corso di *Ingegneria del Software* - Prof. Andrea De Lucia



Object Design Document CardioCare360

Riferimento	Object Design Document v 1.0 2026_ODD_C08_V1.0
Versione	1.0
Data	23/02/2025
Destinatario	Prof. Andrea De Lucia
Presentato da	Ciro Cancellò
Approvato da	/

Indice generale

Revision History.....	3
Team Members.....	4
1. Introduzione.....	5
1.1 Object Design Goals.....	5
1.2 Object Design Trade-Off.....	6
1.3 Linee guida per la documentazione dell'interfaccia.....	6
• Java.....	6
• Spring Boot.....	6
• REST API.....	7
• UML.....	7
• React (Frontend).....	7
• MySQL (Database).....	7
1.4 Definizioni, acronimi ed abbreviazioni.....	8
1.5 Riferimenti.....	10
1.6 Component Off-the-Shelf.....	11
1.7 Design pattern.....	12
1.7.1 Facade.....	12
Applicazione nel sistema.....	12
1.7.2 Adapter.....	13
1.7.3 DAO / Repository.....	14
2. Packages.....	15
2.1 Package cardicare360 (root).....	15
2.2 Package model.....	16
2.3 Package controller.....	16
2.4 Package service.....	16
2.5 Package repository.....	16
2.6 Package security.....	16
2.7 Package dto.....	17
3. Class Interfaces.....	18
3.1 Package model (entity).....	18
3.2 Package controller.....	21
3.3 Package Service.....	22
3.4 Package Repository.....	24
3.5 Package Security.....	24
3.6 Package Response.....	25
3.7 Package Request.....	26
4. Class Diagram.....	27
5. Glossario.....	28

Revision History

Data	Versione	Descrizione	Autore
08/02/2026	0.1	Stesura Object Design Goals	Ciro Canello
09/02/2026	0.2	Stesura Object Design Trade-Off	Ciro Canello
10/02/2026	0.3	Stesura Off-The-Shelf	Ciro Canello
01/04/2026	0.4	Stesura Linee Guida	Ciro Canello
11/02/2026	0.5	Stesura Desgin Patterns	Ciro Canello
13/03/2026	0.6	Stesura packages	Ciro Canello
15/02/2026	0.7	Stesura class interfaces	Ciro Canello
18/02/2026	0.8	Stesura class diagram	Ciro Canello
20/02/2026	0.9	Revisione stile	Ciro Canello
21/02/2026	0.10	Grammar Revision	Ciro Canello
22/02/2026	0.11	Modifica Class Interfaces e Class Diagram	Ciro Canello
23/02/2026	1.0	Revisione e Consegna	Ciro Canello

Team Members

Ruolo	Nome e Cognome	Acronimo	Email
PM	Ciro Cannello	CC	c.cannello@studenti.unisa.it
TM	Ciro Cannello	CC	c.cannello@studenti.unisa.it

1. Introduzione

1.1 Object Design Goals

Rank	ID Design Goal	Descrizione	Origine
1	ODG_1 Robustezza	Il sistema deve garantire un'elevata robustezza nella gestione delle operazioni cliniche. In caso di errori o condizioni anomale (es. indisponibilità del database, dati clinici non integri), il sistema deve essere in grado di intercettare e gestire almeno il 90% delle eccezioni tramite meccanismi di error handling strutturati.	DG_6
2	ODG_2 Sicurezza	Il sistema deve assicurare un elevato livello di sicurezza, garantendo comunicazioni protette tramite protocollo HTTPS, gestione sicura dell'autenticazione (JWT) e autorizzazioni basate sui ruoli (Paziente, Medico, Amministratore).	DG_3
3	ODG_3 Manutenibilità	Il sistema deve essere facilmente manutenibile grazie a una chiara separazione in package (controller, service, repository, model), naming convention coerenti e aderenza alle linee guida di progettazione orientata agli oggetti.	DG_8, DG_13, DG_14
4	ODG_4 Riusabilità	Il sistema deve favorire la riusabilità delle componenti, in particolare dei servizi applicativi e delle classi di dominio, garantendo che almeno il 90% della business logic sia riutilizzabile tramite l'adozione di design pattern (DAO, Facade, Adapter).	DG_13
5	ODG_5 Tempo di risposta	Il sistema deve rispondere alle richieste HTTP del frontend in meno di un secondo nel 95% dei casi, garantendo un'esperienza fluida per Pazienti, Medici e Amministratori.	DG_12

1.2 Object Design Trade-Off

Trade-off	Descrizione
Tempi di risposta vs Sicurezza	Per garantire un elevato livello di sicurezza, il sistema utilizza protocolli HTTPS, token JWT e controlli di autorizzazione basati sui ruoli. Questi meccanismi, pur introducendo un leggero overhead computazionale, assicurano la protezione dei dati sanitari sensibili. Nonostante ciò, il sistema mantiene tempi di risposta inferiori al secondo nel 90% dei casi.
Tempi di risposta vs Robustezza	La robustezza del sistema richiede controlli approfonditi sugli input, validazioni, gestione delle eccezioni e verifiche di consistenza dei dati clinici. Queste operazioni possono aumentare i tempi di elaborazione, ma garantiscono un funzionamento affidabile. I tempi di risposta rimangono comunque inferiori ai 2 secondi nell'85% dei casi.
Costi di sviluppo vs Utilizzo COTS	Per contenere i costi e ridurre i tempi di sviluppo, il sistema fa uso di componenti COTS (es. framework Spring Boot, librerie di sicurezza, strumenti di persistenza). Questo approccio riduce il carico di sviluppo interno, pur mantenendo elevati standard qualitativi.

1.3 Linee guida per la documentazione dell'interfaccia

Le linee guida per la documentazione delle interfacce definiscono le convenzioni che gli sviluppatori devono seguire durante la progettazione e lo sviluppo delle componenti del sistema. Tali convenzioni garantiscono uniformità, chiarezza e coerenza tra le diverse parti dell'applicativo CardioCare360.

In particolare, per le tecnologie utilizzate nel progetto saranno seguite le indicazioni sul coding style e sulle best practices riportate nei seguenti riferimenti:

• Java

- Convenzioni del *Google Java Style Guide*
- Principi di progettazione di *Effective Java*
- Naming chiaro e coerente per classi, metodi e attributi
- Uso corretto di eccezioni, generics e immutabilità

• Spring Boot

- Best practices ufficiali della documentazione Spring
- Organizzazione dei package secondo architettura a livelli (controller, service, repository, model)

- Gestione corretta delle dipendenze tramite Maven
- Utilizzo di DTO per separare API e domain model

• **REST API**

- Naming semantico delle risorse (es. `/api/appuntamenti`, `/api/terapie`)
- Utilizzo appropriato dei metodi HTTP (GET, POST, PUT, DELETE)
- Codici di stato coerenti (200, 201, 400, 404, 500)
- Risposte JSON uniformi e documentate

• **UML**

- Convenzioni standard UML 2.x per Class Diagram, Sequence Diagram e Activity Diagram
- Notazione chiara per relazioni, cardinalità, visibilità e stereotipi

• **React (Frontend)**

- Convenzioni di stile React: componenti funzionali, hooks, separazione logica/presentazione
- Naming coerente dei componenti (PascalCase)
- Gestione dello stato tramite `useState`, `useEffect` o eventuale state management
- Struttura delle cartelle secondo best practices (components, pages, services)

• **MySQL (Database)**

- Naming coerente delle tabelle (`snake_case`)
- Tipi di dato appropriati (VARCHAR, INT, DATE, TIMESTAMP)
- Uso corretto delle chiavi primarie, esterne e vincoli di integrità
- Normalizzazione delle entità secondo schema ufficiale CardioCare360

1.4 Definizioni, acronimi ed abbreviazioni

Termine	Definizione
Package	Raggruppamento logico di classi, interfacce o componenti correlati, utilizzato per organizzare il codice in modo modulare e coerente.
Design Pattern	Soluzioni generiche e riutilizzabili a problemi ricorrenti di progettazione, impiegate per migliorare manutenibilità, estendibilità e riuso del codice.
Interfaccia	Insieme delle signature delle operazioni offerte da una classe o componente, utilizzata per definire contratti e separare implementazione e utilizzo.
DAO (Data Access Object)	Pattern che incapsula l'accesso ai dati e separa la logica applicativa dalla persistenza.
Repository	Implementazione del DAO nel contesto Spring Data, che fornisce metodi CRUD automatici.
DTO (Data Transfer Object)	Oggetto utilizzato per trasferire dati tra livelli dell'applicazione o tra client e server, evitando di esporre direttamente le entità.
REST (Representational State Transfer)	Stile architetturale per la progettazione di API basate su risorse, metodi HTTP e rappresentazioni JSON.
UML (Unified Modeling Language)	Linguaggio di modellazione standard utilizzato per rappresentare classi, relazioni, comportamenti e flussi del sistema.
React	Libreria JavaScript per la costruzione di interfacce utente basate su componenti riutilizzabili.
Component	Unità funzionale di React che incapsula logica e presentazione.

Termine	Definizione
Hook	Funzioni speciali di React (es. <code>useState</code> , <code>useEffect</code>) che permettono di gestire stato e ciclo di vita nei componenti funzionali.
MySQL	Sistema di gestione di database relazionale utilizzato per memorizzare i dati persistenti del sistema.
Entity	Classe Java che rappresenta una tabella del database e viene gestita tramite JPA/Hibernate.
Service	Componente che contiene la logica applicativa del sistema.
Controller	Componente che espone endpoint REST e gestisce le richieste HTTP provenienti dal frontend.
JWT (JSON Web Token)	Token crittografico utilizzato per autenticazione e autorizzazione nelle API REST.
API (Application Programming Interface)	Interfaccia che permette la comunicazione tra frontend e backend tramite richieste HTTP.
HTTP Method	Operazioni standard delle API REST: GET, POST, PUT, DELETE.
JSON (JavaScript Object Notation)	Formato di scambio dati utilizzato nelle comunicazioni tra frontend e backend.
Exception	Evento anomalo gestito tramite meccanismi di error handling.
COTS (Commercial Off-The-Shelf)	Componenti software/hardware già pronti e disponibili sul mercato, integrabili nel sistema senza sviluppo interno.

1.5 Riferimenti

- Bruegge, B., & Dutoit, A. H.** *Object-Oriented Software Engineering – Using UML, Patterns, and Java*, Third Edition. Testo di riferimento per analisi, design, UML e pattern.
- **Documentazione Spring Boot** Linee guida ufficiali per architettura, configurazione, sicurezza, REST controller e best practices.
<https://spring.io/projects/spring-boot> ([spring.io in Bing](#))
 - **Documentazione React** Best practices per componenti, hooks, gestione dello stato e struttura del frontend. <https://react.dev>
 - **Documentazione MySQL** Specifiche tecniche per schema, tipi di dato, vincoli, normalizzazione e ottimizzazione query. <https://dev.mysql.com/doc/>
 - **Specifiche UML (OMG)** Convenzioni standard per Class Diagram, Sequence Diagram, Activity Diagram. <https://www.omg.org/spec/UML/>
 - **Google Java Style Guide** Linee guida ufficiali per stile, naming, formattazione e convenzioni Java.
 - **Effective Java – Joshua Bloch** Principi avanzati di progettazione orientata agli oggetti e best practices Java.
 - **Linee guida REST** Convenzioni per naming delle risorse, metodi HTTP, codici di stato e formati JSON.
 - **Proposta Progetto CardioCare360** Documento ufficiale di riferimento per requisiti, attori, funzionalità e vincoli del sistema.
 - **RAD – 2026_RAD_CardioCare360_V1.0** Documento di analisi dei requisiti funzionali e non funzionali.
 - **SDD – 2026_SDD_CardioCare360_V1.0** Documento di specifica del design del sistema.
 - **ODD – 2026_ODD_CardioCare360_V1.0** Documento di Object Design (il presente documento).

1.6 Component Off-the-Shelf

Nella realizzazione della piattaforma CardioCare360 verranno utilizzati componenti *Off-the-Shelf* già disponibili, al fine di ridurre i tempi di sviluppo, garantire stabilità e mantenere invariati i costi. L'adozione di tali componenti consente di concentrarsi sulla logica applicativa specifica del dominio sanitario, sfruttando strumenti consolidati e ampiamente supportati.

- **Backend – Spring Boot** Per la progettazione del lato backend verrà utilizzato il framework *Spring Boot*, che mette a disposizione librerie standard per la gestione delle API REST, la sicurezza, la persistenza dei dati e la configurazione del progetto. L'utilizzo di moduli come Spring Web, Spring Security e Spring Data JPA permette di ridurre significativamente il lavoro necessario per implementare funzionalità comuni.
- **Frontend – React** Per la progettazione del lato frontend verrà utilizzato *React*, una libreria JavaScript che offre componenti riutilizzabili, gestione dello stato e un ecosistema maturo di strumenti per la costruzione dell'interfaccia utente. L'impiego di librerie già disponibili (React Router, Axios, componenti UI) consente di velocizzare lo sviluppo e mantenere un'interfaccia moderna e responsiva.
- **Database – MySQL** I dati verranno memorizzati all'interno di un DBMS *MySQL*, che garantisce affidabilità, performance e pieno supporto alle transazioni. L'interazione con il database avverrà tramite Spring Data JPA, che fornisce un layer di astrazione per la gestione delle entità e delle query, riducendo la complessità della persistenza.

Tutte le tecnologie adottate sono disponibili gratuitamente e supportate da una vasta comunità, permettendo di ridurre il carico di sviluppo senza incidere sui costi complessivi del progetto.

1.7 Design pattern

1.7.1 Facade

Il pattern *Facade* viene utilizzato per semplificare l'accesso ai sottosistemi del backend, fornendo un unico punto di ingresso alle funzionalità principali. Nel sistema CardioCare360, ogni funzionalità applicativa (gestione appuntamenti, gestione disponibilità, gestione referti, gestione terapie ecc..) espone un'interfaccia denominata **Service**, che rappresenta il punto di accesso unificato ai metodi necessari.

L'utilizzo di tali interfacce consente di:

- ridurre l'accoppiamento tra Controller e logica applicativa
- migliorare la manutenibilità del codice
- permettere la sostituzione o estensione delle implementazioni senza modificare i Controller
- garantire una chiara separazione tra livelli dell'architettura

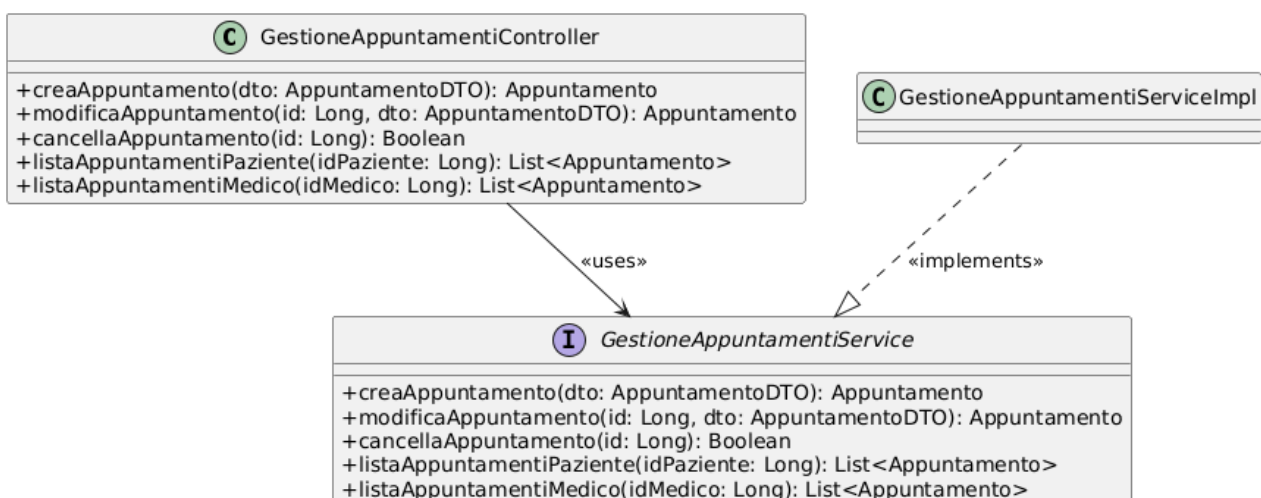
Di seguito è riportato un esempio dell'applicazione del pattern *Facade* nel modulo di gestione degli appuntamenti.

Applicazione nel sistema

Nel modulo **Gestione Appuntamenti**, il pattern Facade è implementato tramite:

- **GestioneAppuntamentiController** → espone gli endpoint REST.
- **GestioneAppuntamentiService** → interfaccia che definisce le operazioni principali.
- **GestioneAppuntamentiServiceImpl** → classe che implementa la logica applicativa.

Il Controller utilizza esclusivamente l'interfaccia **GestioneAppuntamentiService**, senza conoscere i dettagli dell'implementazione. Questo garantisce un'architettura **pulita, estensibile e facilmente testabile**.



1.7.2 Adapter

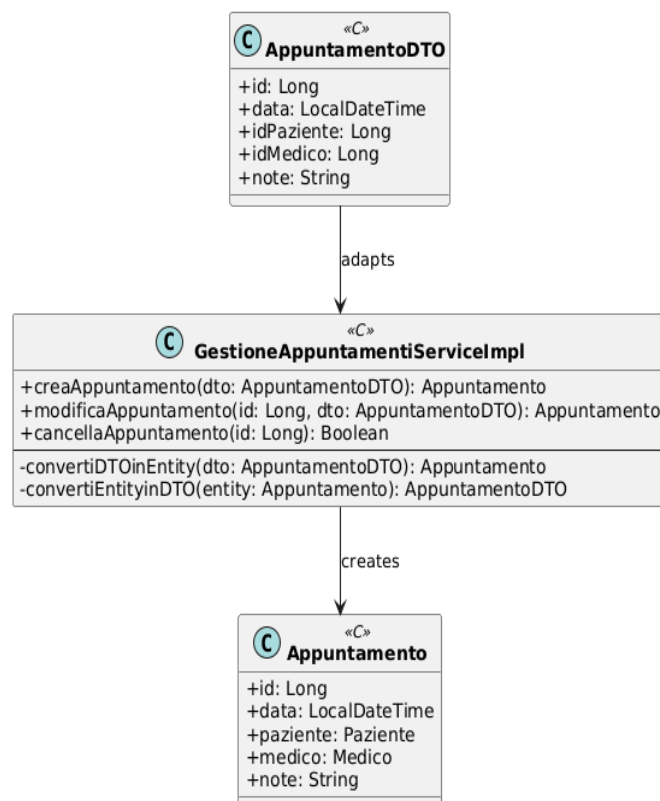
Il pattern Adapter viene utilizzato per convertire oggetti provenienti da un formato esterno (DTO inviati dal frontend React) nelle corrispondenti entità del dominio utilizzate dal backend Spring Boot. Nel sistema CardioCare360 il pattern è implementato tramite metodi di conversione presenti nei **Service** e nei **Mapper**, anche se non è definita una classe esplicitamente denominata “Adapter”.

L'utilizzo di questo pattern consente di:

- separare la logica di conversione dalla logica applicativa
- mantenere i Controller puliti e focalizzati sulle API
- ridurre l'accoppiamento tra livelli
- garantire coerenza nella trasformazione dei dati
- facilitare modifiche future ai formati di input/output

Di seguito è riportato un esempio dell'applicazione del pattern Adapter nel modulo di gestione degli appuntamenti, dove i metodi di conversione trasformano `AppuntamentoDTO` in `Appuntamento` e viceversa.

Pattern Adapter implicito - Modulo Gestione Appuntamenti



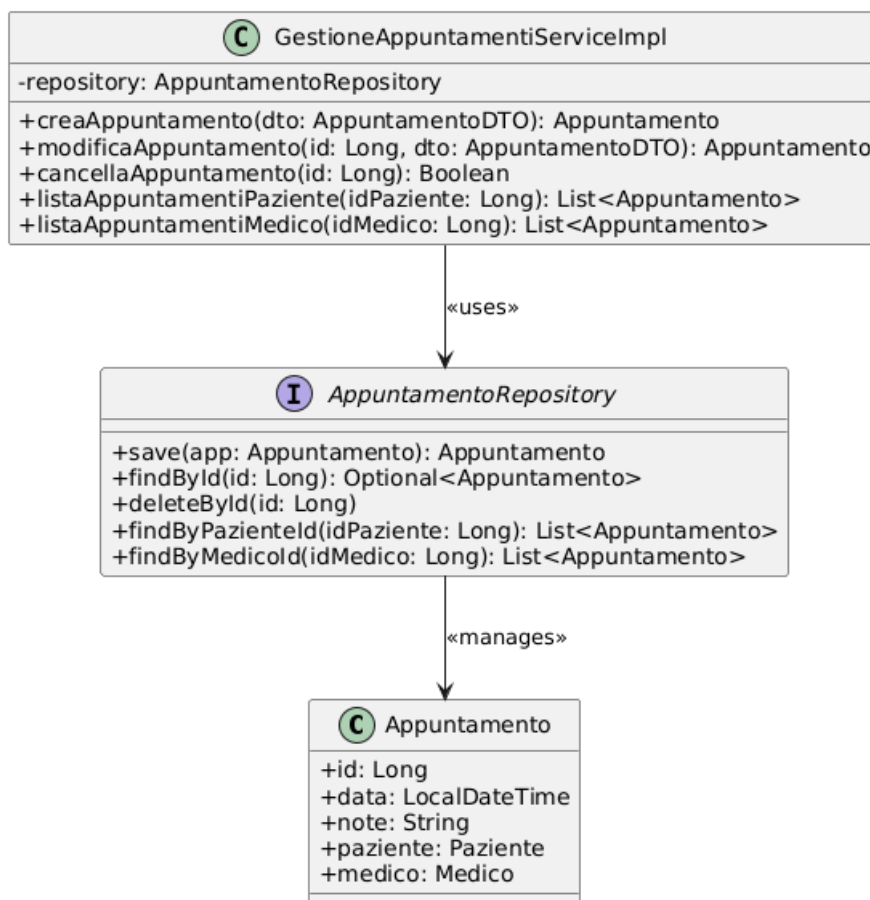
1.7.3 DAO / Repository

Il pattern *Data Access Object (DAO)* è un design pattern architetturale che fornisce un'interfaccia astratta per l'accesso ai dati, permettendo al sistema di interagire con il database senza esporre i dettagli di implementazione. Nel contesto di CardioCare360, il DAO viene implementato tramite il modulo **Repository** di Spring Data JPA, che consente di mappare le entità del dominio alle tabelle del database MySQL e di eseguire operazioni CRUD in modo semplice e sicuro.

L'utilizzo del pattern DAO/Repository permette di:

- separare la logica applicativa dalla logica di persistenza
- ridurre l'accoppiamento tra livelli
- migliorare la manutenibilità del codice
- sfruttare query automatiche e personalizzate tramite Spring Data
- garantire un accesso ai dati efficiente e coerente

Nel sistema CardioCare360 sono stati implementati diversi DAO/Repository per la gestione delle entità principali (Paziente, Medico, Appuntamento, Disponibilità, Referto). Di seguito è riportato un esempio relativo alla gestione degli appuntamenti.



2. Packages

In questa sezione viene mostrata la suddivisione del sistema in package, conforme a quanto definito nel documento di System Design. La struttura riflette le scelte architetturali adottate, basate sul modello **Three Tier**, che suddivide l'applicazione CardioCare360 nei seguenti livelli:

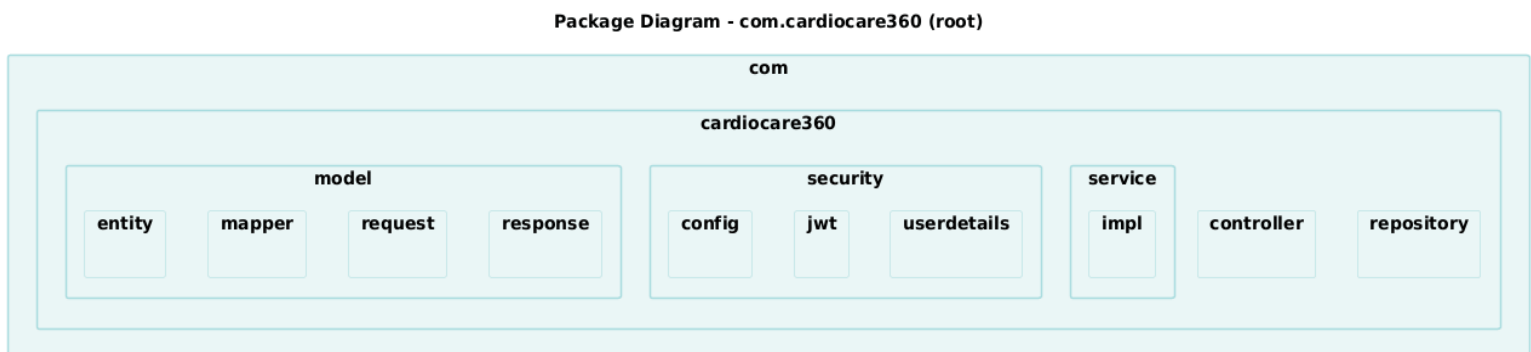
- **Presentation Layer** → React
- **Application/Business Layer** → Spring Boot (Controller, Service, Security, DTO, Exception)
- **Data Layer** → Spring Data JPA + MySQL

La seguente suddivisione descrive i principali package del backend e le directory del frontend.

2.1 Package cardiacare360 (root)

Il package root contiene tutti i package principali del backend Spring Boot. Rappresenta la struttura logica dell'applicazione lato server.

Di seguito è riportata la rappresentazione grafica della suddivisione in package.



2.2 Package model

Il package `model` contiene le **entità del dominio** e le **classi correlate** che rappresentano la struttura dati persistente e le conversioni tra livelli applicativi. È organizzato in sottopacchetti per garantire separazione logica e coerenza con l'architettura **Three-Tier** del sistema.

Struttura reale:

- **model.entity** → entità JPA che mappano le tabelle del database:
 - Utente, Paziente, Medico, Amministratore
 - Appuntamento, DisponibilitaMedico, SlotOrario
 - Esame, Referto, DocumentoPDF
 - Terapia, Farmaco
 - Parametro, ParametroClinico, StoricoParametri, SogliaParametro
 - Messaggio, Conversazione, Notifica

Le entità sono annotate con `@Entity` e gestite tramite **Spring Data JPA**, con relazioni `@OneToMany`, `@ManyToOne` e `@OneToOne` coerenti con lo schema logico del database.

- **model.mapper** → classi di mapping per la conversione **Entity** ↔ **DTO**, utilizzate nei layer `service` e `controller` per separare la logica di dominio dalla rappresentazione esterna.
- **model.request** → oggetti di input per le API REST, utilizzati per la validazione dei dati in ingresso e la creazione di nuove entità (es. `AppuntamentoRequest`, `ParametroClinicoRequest`, `TerapiaRequest`).
- **model.response** → oggetti di output per le API REST, utilizzati per restituire dati strutturati al frontend (es. `AppuntamentoResponse`, `RefertoResponse`, `ParametroResponse`).

Nota di aggiornamento

L'introduzione dell'entità **Parametro** ha permesso di:

- separare la definizione del tipo di parametro (es. pressione, glicemia, saturazione) dai valori rilevati (`ParametroClinico`);
- migliorare la normalizzazione del database;
- semplificare la gestione delle soglie (`SogliaParametro`) e dello storico (`StoricoParametri`);
- garantire maggiore coerenza tra livello di persistenza e logica applicativa.

2.3 Package controller

Il package `controller` contiene i controller REST che espongono le API verso il frontend React. Ogni controller gestisce un modulo funzionale (Appuntamenti, Pazienti, Medici, Autenticazione, ecc.).

2.4 Package service

Il package `service` rappresenta il livello applicativo. Struttura reale:

- `service` → interfacce Service (pattern Facade)
- `service.impl` → implementazioni della logica applicativa

2.5 Package repository

Il package `repository` contiene le interfacce DAO basate su Spring Data JPA. Gestisce l'accesso al database tramite metodi CRUD e query personalizzate.

2.6 Package security

Il package `security` contiene tutte le componenti relative alla sicurezza. Struttura reale:

- `security.config` → configurazione Spring Security
- `security.jwt` → gestione token JWT
- `security.userdetails` → gestione utenti e ruoli

2.7 Package dto

Il package `dto` contiene i Data Transfer Object utilizzati per la comunicazione tra backend e frontend.

Struttura del frontend

Il frontend è sviluppato in React e organizzato nelle seguenti directory:

cardiocare360-frontend

src/

components/ → componenti riutilizzabili

pages/ → pagine principali (Paziente, Medico, Admin)

services/ → chiamate API REST

context/ → stato globale (autenticazione)

utils/ → funzioni di utilità

assets/ → immagini, icone, risorse statiche

public/

index.html

package.json

3. Class Interfaces

3.1 Package model (entity)

Il package `com.cardiocare360.model.entity` contiene le entità del dominio, mappate sulle tabelle MySQL tramite annotazioni JPA. Le classi rappresentano gli oggetti principali del sistema CardioCare360 e le loro relazioni.

Nome Classe	Descrizione	Attributi principali	Metodi significativi	Invariante di classe
Utente	Rappresenta l'utente del sistema, con credenziali e ruoli.	id: Long, email: String, password: String, ruoli: List<Ruolo>	n/a	email $\neq$ null $\wedge$ password $\neq$ null
Admin	Utente con privilegi amministrativi.	id: Long, utente: Utente	n/a	utente $\neq$ null
Paziente	Utente sanitario registrato nel sistema.	id: Long, nome: String, cognome: String, dataNascita: LocalDate, telefono: String, utente: Utente	n/a	nome $\neq$ null $\wedge$ cognome $\neq$ null
Medico	Rappresenta un medico con specializzazione e disponibilità.	id: Long, nome: String, cognome: String, specializzazione: String, utente: Utente	n/a	specializzazione $\neq$ null
Appuntamento	Collega paziente e medico in una data e ora.	id: Long, data: LocalDate, ora: LocalTime, stato: String, paziente: Paziente, medico: Medico	n/a	data $\geq$ oggi
DisponibilitaMedico	Fascia oraria di disponibilità del medico.	id: Long, data: LocalDate, oraInizio: LocalTime, oraFine: LocalTime, medico: Medico	n/a	oraFine $>$ oraInizio
SlotOrario	Segmento orario prenotabile.	id: Long, oraInizio: LocalTime, oraFine: LocalTime, disponibilita: DisponibilitaMedico	n/a	oraFine $>$ oraInizio
DocumentoPDF	Documento allegato a referti o esami.	id: Long, nomeFile: String, contenuto: byte[], appuntamento: Appuntamento	n/a	nomeFile $\neq$ null
Esame	Rappresenta un	id: Long, tipoEsame:	n/a	tipoEsame $\neq$ null

Nome Classe	Descrizione	Attributi principali	Metodi significativi	Invariante di classe
	esame diagnostico.	String, dataEsame: LocalDate, oraEsame: LocalTime, paziente: Paziente, medico: Medico, stato: String		
Referto	Documento clinico associato a un esame.	id: Long, diagnosi: String, note: String, appuntamento: Appuntamento	n/a	diagnosi ≠ null
Farmaco	Informazioni sui medicinali prescritti.	id: Long, nome: String, principioAttivo: String	n/a	nome ≠ null
Terapia	Prescrizione medica per un paziente.	id: Long, farmaco: String, dosaggio: String, dataInizio: LocalDate, dataFine: LocalDate, paziente: Paziente	n/a	dataFine ≥ dataInizio
Conversazione	Thread di messaggi tra paziente e medico.	id: Long, paziente: Paziente, medico: Medico, messaggi: List<Messaggio>	n/a	paziente ≠ null ∧ medico ≠ null
Messaggio	Comunicazione tra paziente e medico.	id: Long, contenuto: String, dataInvio: LocalDateTime, mittente: Utente, conversazione: Conversazione	n/a	contenuto ≠ null
Notifica	Promemoria o avviso per l'utente.	id: Long, messaggio: String, dataInvio: LocalDateTime, utente: Utente	n/a	messaggio ≠ null
Parametro	Definisce il tipo di parametro clinico monitorato (es. pressione, glicemia, temperatura).	id: Long, nome: String, tipo: String, unitaMisura: String	n/a	nome ≠ null ∧ unitaMisura ≠ null
ParametroClinico	Rappresenta i valori rilevati per ciascun parametro del paziente.	id: Long, nome: String, unitaMisura: String, dataRilevazione: LocalDateTime, tipo: String, paziente: Paziente, alert: Boolean, battiti: Double, glicemia: Double, peso: Double, pressioneDiastolica: Double, pressioneSistolica: Double	n/a	dataRilevazione ≠ null

Nome Classe	Descrizione	Attributi principali	Metodi significativi	Invariante di classe
SogliaParametro	Definisce i limiti di normalità per ciascun parametro clinico del paziente.	Double, saturazione: Double, temperatura: Double id: Long, parametro: Parametro, valoreMin: Double, valoreMax: Double, paziente: Paziente	n/a	valoreMax > valoreMin
StoricoParametri	Registra la cronologia dei valori rilevati per ciascun parametro.	id: Long, paziente: Paziente, parametro: Parametro, valore: Double, dataRilevazione: LocalDateTime	n/a	valore ≠ null

3.2 Package controller

Il package `com.cardiocare360.controller` contiene i controller REST che espongono le API verso il frontend React. Ogni controller gestisce un insieme di funzionalità specifiche del dominio applicativo CardioCare360.

Nome Classe	Descrizione	Endpoint	Dipendenze
AdminController	Operazioni amministrative	/admin/*	AdminService
AppuntamentoController	Gestione appuntamenti	/appuntamenti/*	AppuntamentoService
AuthController	Autenticazione JWT	/auth/*	AuthService, JwtTokenProvider
ConversazioneController	Chat paziente-medico	/conversazioni/*	ConversazioneService
DisponibilitaController	Disponibilità medici	/disponibilita/*	DisponibilitaMedicoService
EsameController	Esami diagnostici	/esami/*	EsameService
FarmacoController	Farmaci	/farmaci/*	FarmacoService
MedicoController	Operazioni sui medici	/medici/*	MedicoService
MessaggioController	Messaggi chat	/messaggi/*	MessaggioService
NotificaController	Notifiche utenti	/notifiche/*	NotificaService
ParametroController	Tipi di parametro (tabella parametro)	/parametri-tipo/*	ParametroService
ParametroClinicoController	Valori clinici rilevati	/parametri/*	ParametroClinicoService
SogliaParametroController	Soglie min/max parametri	/soglie/*	SogliaParametroService
StoricoParametriController	Storico valori parametri	/storico-parametri/*	StoricoParametriService
ParametroClinicoMedicoController	Associazione medico ↔ parametri	/parametri-medico/*	ParametroClinicoMedicoService
PazienteController	Gestione pazienti	/pazienti/*	PazienteService
RefertoController	Referti medici	/referti/*	RefertoService
TerapiaController	Terapie	/terapie/*	TerapiaService
UtenteController	Operazioni sugli utenti	/utenti/*	UtenteService

3.3 Package Service

Il package `com.cardiocare360.service` contiene le interfacce del livello applicativo. Ogni service definisce le operazioni di business per un modulo specifico del sistema CardioCare360 e viene implementato nel sotto-package `service.impl`

Classe	Descrizione	Responsabilità	Dipendenze
AppuntamentoService	Logica di business per appuntamenti.	CRUD appuntamenti, ricerca per paziente/medico	AppuntamentoRepository, PazienteService, MedicoService
AuthService	Gestione autenticazione JWT.	Login, registrazione, token.	UtenteRepository, JwtTokenProvider
ConversazioneService	Gestione chat paziente-medico.	Creazione conversazioni, recupero messaggi.	ConversazioneRepository, MessaggioService
DisponibilitaMedicoService	Gestione disponibilità mediche.	Creazione e aggiornamento fasce orarie.	DisponibilitaMedicoRepository, MedicoService
EsameService	Gestione esami diagnostici.	CRUD esami, aggiornamento stato.	EsameRepository, MedicoService, PazienteService
MedicoService	Operazioni sui medici.	Gestione profili, disponibilità, appuntamenti.	MedicoRepository, DisponibilitaMedicoService
MessaggioService	Gestione messaggi.	Invio, ricezione, archiviazione.	MessaggioRepository, UtenteService
NotificaService	Gestione notifiche.	Creazione, invio, lettura promemoria.	NotificaRepository, UtenteService
ParametroClinicoService	Gestione parametri vitali.	CRUD parametri, associazione a pazienti.	ParametroClinicoRepository, SogliaParametroService
PazienteService	Operazioni sui pazienti.	Registrazione, aggiornamento, dati clinici.	PazienteRepository, UtenteService
PdfService	Gestione documenti PDF.	Generazione referti/esami PDF.	DocumentoPDFRepository, RefertoService
RefertoService	Gestione referti medici.	Creazione, aggiornamento, consultazione.	RefertoRepository, AppuntamentoService
SlotOrarioService	Gestione slot orari.	Creazione e gestione	SlotOrarioRepository, DisponibilitaMedicoService

Classe	Descrizione	Responsabilità	Dipendenze
TerapiaService	Gestione terapie.	segmenti orari. CRUD terapie, prescrizioni pazienti.	TerapiaRepository, PazienteService, FarmacoService

3.4 Package Repository

Il package `com.cardiocare360.repository` contiene le interfacce DAO basate su Spring Data JPA, responsabili dell’accesso ai dati e delle operazioni CRUD sulle entità del dominio CardioCare360.

Classe	Descrizione	Entità	Metodi principali
AdminRepository	Dati amministratori.	Admin	findByUtenteId(Long id)
AppuntamentoRepository	Appuntamenti paziente-medico.	Appuntamen to	findByPazienteId(Long id), findByMedicoId(Long id)
ConversazioneRepository	Conversazioni tra utenti.	Conversazi one	findByPazienteId(Long id), findByMedicoId(Long id)
DisponibilitaMedicoRepository	Disponibilità mediche.	Disponibil itaMedico	findByMedicoId(Long id), findByData(LocalDate data)
EsameRepository	Esami diagnostici.	Esame	findByPazienteId(Long id), findByMedicoId(Long id)
FarmacoRepository	Farmaci prescritti.	Farmaco	findByNome(String nome)
MedicoRepository	Dati medici.	Medico	findBySpecializzazione(String spec)
MessaggioRepository	Messaggi conversazioni.	Messaggio	findByConversazioneId(Long id)
NotificaRepository	Notifiche utenti.	Notifica	findByUtenteId(Long id)
ParametroClinicoRepository	Parametri vitali.	ParametroC linico	findByNome(String nome)
PazienteRepository	Dati pazienti.	Paziente	findByUtenteId(Long id)
RefertoRepository	Referti medici.	Referto	findByAppuntamentoId(Long id)
SlotOrarioRepository	Slot orari.	SlotOrario	findByDisponibilitaId(Long id)
TerapiaRepository	Terapie pazienti.	Terapia	findByPazienteId(Long id)
UtenteRepository	Dati utenti.	Utente	findByEmail(String email)

3.5 Package Security

Il package `com.cardiocare360.security` contiene le componenti dedicate alla sicurezza dell'applicazione. Gestisce autenticazione, autorizzazione, generazione e validazione dei token JWT, oltre alla configurazione di Spring Security.

Classe	Descrizione	Ruolo	Dipendenze principali
SecurityConfig	Configura le regole di sicurezza Spring.	Definisce filtri, CORS, ruoli e accessi.	JwtFilter, CustomUserDetailsService
JwtFilter	Filtro per la validazione dei token JWT.	Intercetta le richieste e verifica autenticazione.	JwtUtil, CustomUserDetailsService
JwtUtil	Utility per la gestione dei token JWT.	Genera, valida e decodifica i token.	io.jsonwebtoken
CustomUserDetailsService	Implementa il caricamento dei dati utente.	Recupera credenziali e ruoli da database.	UtenteRepository

3.6 Package Response

Il package `com.cardiocare360.model.response` contiene le classi DTO e Response utilizzate per la comunicazione tra backend e frontend. Queste classi definiscono i dati restituiti dalle API REST del sistema CardioCare360.

Classe	Descrizione	Ruolo	Campi principali
AppuntamentoDTO	Dati di un appuntamento.	Trasferimento dati appuntamento.	id, data, ora, stato, paziente, medico
AuthResponse	Risposta autenticazione JWT.	Restituisce token e info utente.	token, ruolo, email
DisponibilitaEsameResponse	Dati disponibilità per esami.	Restituisce slot e medici disponibili.	data, ora, medico, tipoEsame
EsameDTO	Dati di un esame diagnostico.	Trasferimento dati esame.	id, tipoEsame, dataEsame, stato
MedicoResponse	Dati di un medico.	Restituisce info profilo medico.	id, nome, cognome, specializzazione
MessaggioDTO	Dati di un messaggio.	Trasferimento contenuto chat.	id, contenuto, mittente, dataInvio
NotificaDTO	Dati di una notifica.	Restituisce promemoria o avvisi.	id, messaggio, dataInvio, letto
ParametriRecentiDTO	Ultimi parametri clinici rilevati.	Restituisce valori recenti.	nomeParametro, valore, dataRilevazione
ParametroClinicoStoricoDTO	Storico parametri clinici.	Restituisce serie temporale valori.	nomeParametro, valori, date
PazienteDTO	Dati di un paziente.	Trasferimento info anagrafiche.	id, nome, cognome, telefono
PazienteResponse	Risposta dettagliata paziente.	Restituisce dati clinici e appuntamenti.	id, nome, appuntamenti, terapie
RefertoDTO	Dati di un referto medico.	Trasferimento diagnosi e note.	id, diagnosi, note, appuntamento
UtenteDTO	Dati di un utente.	Trasferimento info base utente.	id, email, ruoli

3.7 Package Request

Il package `com.cardiocare360.model.request` contiene le classi DTO e Request utilizzate per la comunicazione **dal frontend verso il backend**. Queste classi definiscono i dati inviati dalle API REST per operazioni di creazione, aggiornamento o autenticazione.

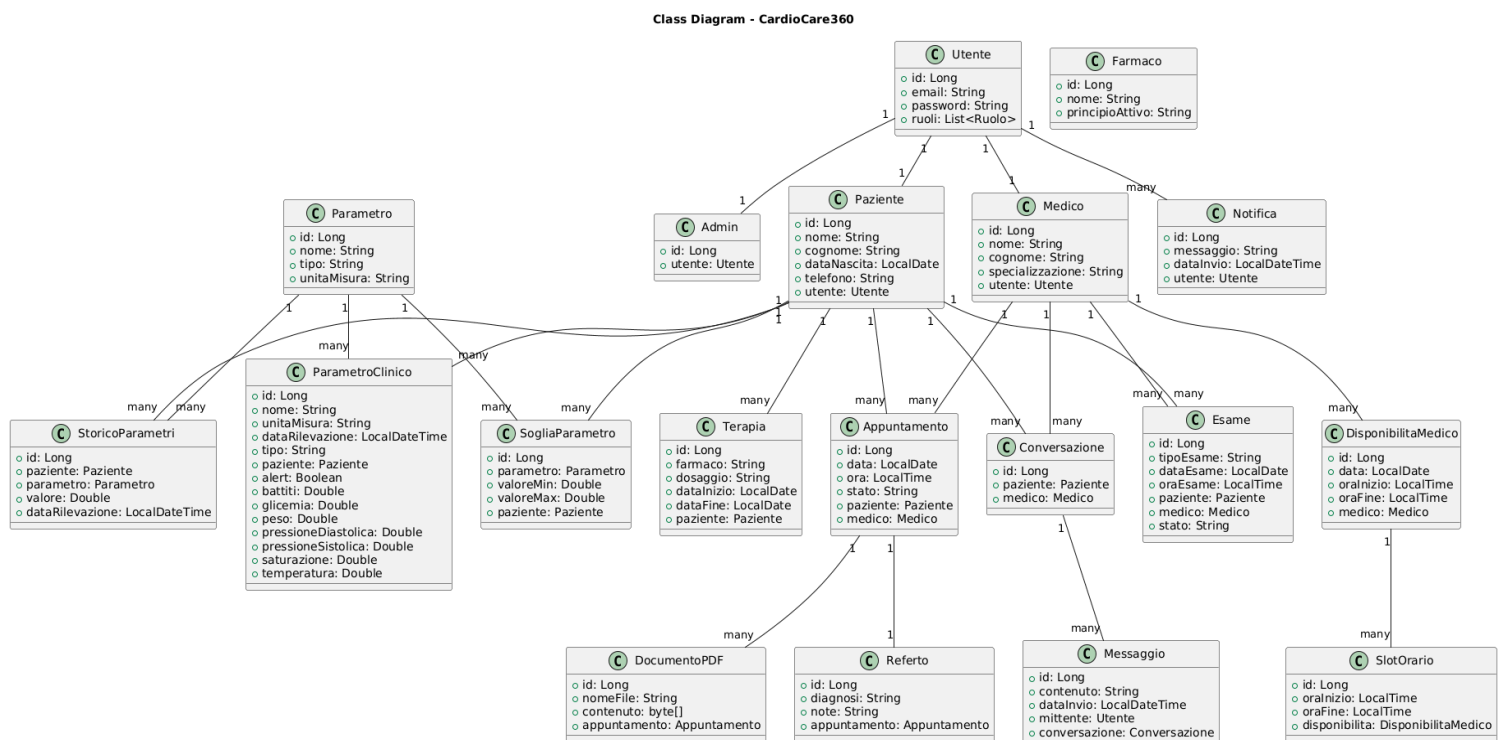
Classe	Descrizione	Ruolo	Campi principali
CambiaPasswordRequest	Richiesta di cambio password utente.	Aggiornamento credenziali.	email, vecchiaPassword, nuovaPassword
CreaMedicoRequest	Richiesta di creazione nuovo medico.	Registrazione medici.	nome, cognome, specializzazione, email, password
LoginRequest	Richiesta di autenticazione utente.	Login e generazione token JWT.	email, password
MedicoUpdateDTO	Dati aggiornamento profilo medico.	Modifica informazioni mediche.	id, telefono, specializzazione
MessaggioRequestDTO	Dati per invio messaggio.	Comunicazione paziente-medico.	contenuto, mittenteId, conversazioneId
NotificaRequestDTO	Dati per creazione notifica.	Invio promemoria o avvisi.	messaggio, utenteId, dataInvio
ParametroClinicoRequest	Dati per inserimento parametro clinico.	Creazione o aggiornamento parametri vitali.	nome, unitaMisura, valore
PazienteUpdateDTO	Dati aggiornamento profilo paziente.	Modifica informazioni anagrafiche.	id, telefono, indirizzo
RefertoRequest	Dati per creazione referto medico.	Inserimento diagnosi e note.	appuntamentoId, diagnosi, note
RegisterRequest	Richiesta di registrazione utente.	Creazione nuovo account.	email, password, ruolo

4. Class Diagram

Il diagramma delle classi rappresenta la struttura logica del dominio CardioCare360, evidenziando le principali entità e le loro relazioni. Le classi sono organizzate secondo il modello **Three-Tier** e rispecchiano la mappatura JPA verso il database MySQL. Le relazioni tra le entità riflettono i legami funzionali del sistema:

- **Utente** è la classe base per autenticazione e ruoli (Medico, Paziente, Admin).
- **Medico** e **Paziente** estendono il concetto di utente e sono collegati tramite **Appuntamento**.
- Ogni **Appuntamento** può generare un **Referto**, un **DocumentoPDF** e può essere associato a un **Esame**.
- **DisponibilitaMedico** e **SlotOrario** definiscono le fasce orarie prenotabili.
- **Terapia**, **Farmaco**, **ParametroClinico**, **StoricoParametri** e **SogliaParametro** modellano la parte clinica del sistema.
- **Conversazione** e **Messaggio** gestiscono la comunicazione tra paziente e medico.
- **Notifica** consente l'invio di promemoria e avvisi agli utenti.

Il diagramma mostra le relazioni di tipo **uno-a-molti**, **molti-a-uno** e **uno-a-uno** tra le entità principali, garantendo coerenza con il modello relazionale del database.



La tabella **notifica** è stata mantenuta nel database perché fa parte del modello concettuale originale del dominio e rappresenta una funzionalità prevista per l'evoluzione del sistema. Anche se nel prototipo attuale non è stata ancora implementata, la sua presenza non crea incoerenze e permette di estendere facilmente il sistema in futuro con notifiche automatiche per appuntamenti, terapie o messaggi. In un progetto modulare come CardioCare360 è preferibile mantenere le entità del dominio che possono essere utili nelle versioni successive, evitando modifiche strutturali al database.

5. Glossario

Sigla/Termine	Definizione
CardioCare360	Sistema software per la gestione di appuntamenti, referti, terapie e disponibilità mediche, rivolto a pazienti, medici e amministratori.
Paziente	Utente del sistema che può prenotare appuntamenti, visualizzare referti, consultare terapie e ricevere notifiche.
Medico	Utente del sistema che può gestire disponibilità, appuntamenti, referti e terapie dei pazienti.
Amministratore	Utente con privilegi avanzati che gestisce medici, pazienti e supervisione generale del sistema.
Appuntamento	Incontro programmato tra paziente e medico, con data, ora e stato.
Disponibilità Medico	Fasce orarie in cui un medico è disponibile per ricevere appuntamenti.
Referto	Documento clinico redatto dal medico contenente diagnosi e informazioni sanitarie del paziente.
Terapia	Trattamento prescritto dal medico, con descrizione, data di inizio e fine.
Notifica	Messaggio inviato al paziente o al medico per ricordare appuntamenti, aggiornamenti o eventi rilevanti.
Backend	Parte del sistema che gestisce logica applicativa, sicurezza, accesso ai dati e API REST (Spring Boot).
Frontend	Interfaccia utente con cui interagiscono pazienti, medici e amministratori (React).
MySQL	

Sigla/Termine	Definizione
	DBMS relazionale utilizzato per la persistenza dei dati di CardioCare360.
DBMS	Sistema software che gestisce la creazione, manipolazione e interrogazione dei database.
Package	Raggruppamento logico di classi con responsabilità affini (model, controller, service, repository, security, dto, exception).
Entity (Model)	Classe che rappresenta un'entità persistente del dominio (Paziente, Medico, Appuntamento, ecc.).
DTO (Data Transfer Object)	Oggetto utilizzato per trasferire dati tra frontend e backend, senza logica applicativa.
Controller	Classe che espone le API REST e gestisce le richieste provenienti dal client.
Service	Classe che implementa la logica di business del sistema.
Repository	Interfaccia che gestisce l'accesso ai dati tramite Spring Data JPA.
Security	Insieme di componenti che gestiscono autenticazione, autorizzazione e validazione dei token JWT.
JWT (JSON Web Token)	Token utilizzato per autenticare e autorizzare gli utenti nelle richieste al backend.
Facade	Pattern che fornisce un'interfaccia semplificata a un insieme di sottosistemi complessi.
Adapter	Pattern strutturale che permette l'interoperabilità tra interfacce diverse.
DAO	Pattern architetturale per l'accesso ai dati, sostituito in CardioCare360

Sigla/Termine	Definizione
	da Spring Data JPA.
REST API	Interfacce HTTP che permettono la comunicazione tra frontend e backend.
COTS (Component Off-The-Shelf)	Componenti software/hardware già pronti e acquistabili, utilizzati per accelerare lo sviluppo.